Projet MLSF : résolution du jeu Meowdoku

Question 3 : résolution et restitution d'une grille

1 Du fichier texte à la solution

La fonction `solve_meowdoku` du fichier `meowdoku.py` reçoit le chemin du fichier texte contenant la grille et celui du fichier résultat. Elle réutilise les programmes des deux premières questions :

1. **Lecture :** `read_meowdoku` transforme le fichier en une matrice de régions. Le texte original est conservé pour être recopié dans le résultat.
2. **Conversion et résolution :** `meowdoku_to_dimacs` écrit les contraintes dans un fichier DIMACS temporaire. `read_dimacs` récupère les clauses, puis `dpll` cherche une valuation qui les satisfait. Si aucune solution n'existe, la fonction renvoie `None` sans écrire de résultat.
3. **Décodage et vérification :** la valuation est complétée, puis les variables vraies sont converties en coordonnées de chats. Le programme vérifie les clauses SAT et les règles du jeu.
4. **Restitution :** le fichier de sortie contient l'entrée originale, les coordonnées du chat de chaque région, puis une grille où `*` représente un chat et `.` une case vide. La fonction renvoie aussi le dictionnaire `{région: (ligne, colonne)}`.

2 Des variables vraies aux coordonnées des chats

La partie centrale de cette question consiste à interpréter le modèle renvoyé par le solveur. Chaque variable correspond à une case : si elle vaut vrai, cette case contient un chat. Pour une grille de m colonnes, la numérotation définie à la question 1 est

$$v = (i-1)m + j,$$

où i et j sont la ligne et la colonne, numérotées à partir de 1. Pour retrouver ces coordonnées, on inverse cette relation :

$$i = \left\lfloor \frac{v-1}{m} \right\rfloor + 1, \qquad j = ((v-1) \bmod m) + 1.$$

Le code correspondant est :

```
positions = [((v - 1) // m + 1, (v - 1) % m + 1)
             for v, value in model.items() if value]
```

- `model.items()` parcourt les couples formés du numéro de variable `v` et de sa valeur booléenne `value`.
- `if value` conserve uniquement les variables vraies, donc les cases contenant un chat.
- `(v - 1) // m + 1` donne la ligne : la division entière compte les lignes complètes qui précèdent la case.
- `(v - 1) % m + 1` donne la colonne : le reste de la division indique la position dans la ligne.

On soustrait 1 avant le calcul pour travailler à partir de zéro, puis on ajoute 1 aux résultats pour obtenir des coordonnées commençant à 1. Par exemple, avec $m = 5$ colonnes et $v = 8$:

$$i = 7 \,//\, 5 + 1 = 2, \qquad j = 7 \bmod 5 + 1 = 3.$$

Si la variable 8 est vraie, un chat est donc placé en $(2, 3)$. La région de ce chat est ensuite obtenue par `grid[i - 1][j - 1]`. Ces positions permettent d'écrire aussi bien les coordonnées par région que les étoiles de la grille résolue.

3 Utilisation

Depuis le dossier du projet :

```
python3 meowdoku.py grille.txt solution.txt --seed 0
```

L'option facultative `-seed` permet de reproduire les choix aléatoires du solveur. Le programme écrit une solution vérifiée dans `solution.txt` si la grille est satisfaisable.